

Guía para crear un módulo CRUD nuevo

Secuencia y patrones para incorporar módulos de catálogo al frontend y backend reutilizando la arquitectura existente del proyecto.

Arquitectura	Endpoint (backend) → Form + Page (frontend) → Ruta
Componentes clave	Resource · Request · Controller · Form · Page · useCrudResource
Objetivo	Migrar módulos de catálogo desde PlaceholderPage manteniendo una estructura uniforme

Documento organizado a partir del contenido proporcionado, conservando sus nombres de archivos, componentes, rutas, configuraciones y ejemplos técnicos.

Guía para crear un módulo CRUD nuevo

Esta guía describe la secuencia para dar de alta un módulo de catálogo (código + nombre + descripción + estado, con crear/editar/buscar/desactivar) reutilizando la arquitectura que ya usan `\Microred\`, `\TipoEstablecimiento\` y `\MetodoMarcacion\`.

No es la única forma de construir una pantalla en este proyecto, pero es la que corresponde a los ~70 módulos de catálogo que quedan por migrar desde `\PlaceholderPage\`.

Secuencia: **`\*\*endpoint (backend)` → `form + page (frontend)` → `rutal\*`.**

Paso 0 — Antes de escribir nada

Confirma que el backend para este recurso ya sigue el patrón estándar del proyecto (26 de 28 controllers ya lo siguen; ver `\UTES-RRHH-Backend\`):

`\- \app\Models/\<Dominio>/\<Entidad>.php\` — ya existe si la tabla está en el schema.

`\- \app\Http\Resources/\<Entidad>Resource.php\` — mapea las columnas de SQL Server (PascalCase) a JSON en **`\*\*camelCase\*`**: `\id\`, `\codigo\`, `\nombre\`, `\descripcion\`, `\activo\`, ...

`\- \app\Http\Requests/\<Entidad>Request.php\` — valida los campos en el nombre de columna real (PascalCase), porque así los espera el modelo Eloquent.

`\- \app\Http\Controllers\Api/\<Entidad>Controller.php\` — `\index\` con

`\->when()\ + \por_pagina\` (clamp 1-100, default 15) devolviendo

`\<Entidad>Resource::collection(...)\`; `\store\` `\show\` `\update\` devuelven

`\<Entidad>Resource\`; `\destroy\` hace baja lógica y responde

`\['mensaje' => '...']\`.

`\- Ruta registrada en \routes/api.php\` con `\Route::apiResource(...)\`.

Si el controller de tu entidad todavía devuelve el modelo crudo (sin Resource), normalízalo primero siguiendo cualquiera de los 26 controllers ya conformes como plantilla (`\GrupoOcupacionalController\` es un buen ejemplo corto). El resto de esta guía asume que el frontend recibe **`\*\*siempre\*`** `\id\` `\codigo\` `\nombre\` `\descripcion\` `\activo\` en camelCase — si el backend no cumple eso, el hook `\useCrudResource\` seguirá funcionando, pero tendrás que escribir un `\mapToForm\` más largo para compensar la inconsistencia en vez de arreglarla.

Paso 1 — Form específico del recurso

Ubicación: junto al `\Page\` que lo va a usar (`\src/pages/\<dominio>/\<Entidad>Form.jsx\`), no en `\components/ui\` — el form es específico del recurso, los átomos que usa (`\Field\`, `\Select\`, `\Textarea\`) son los genéricos.

`\jsx`

`// src/pages/\<dominio>/\<Entidad>Form.jsx`

`import { Hash, Building2, FileText } from 'lucide-react';`

```
import Field from '../components/ui/Field';

export default function \<Entidad>Form({ form, setForm, errors }) {
  return (
    <>
    \<Field form={form} setForm={setForm} errors={errors} name="\<Entidad>Codigo" label="Código" icon={Hash} />
    \<Field form={form} setForm={setForm} errors={errors} name="\<Entidad>Nombre" label="Nombre" icon={Building2} />
    \<Field form={form} setForm={setForm} errors={errors} name="\<Entidad>Descripcion" label="Descripción" icon={FileText} />
    \</>
  );
}

\\`
```

Componentes disponibles en `\src/components/ui/\` para construir el form:

| Componente | Cuándo usarlo |

| ---|---|

| \Field\ | Texto simple (input) |

| \Select\ | Lista fija o catálogo relacionado (\options={{value, label}}\)|

| \Textarea\ | Texto largo (descripciones extensas, observaciones) |

Los tres reciben las mismas props (\form\, \setForm\, \errors\, \name\, \label\, \icon\ opcional) y se leen/escriben solos contra \form[name]\ y \errors[name]?.[0]\ — nunca escribas \onChange\ a mano.

Si necesitas dos campos lado a lado, envuélvelos en un `\\<div className="grid grid-cols-2 gap-4">\` directamente en el Form (es la única clase de Tailwind que sigue siendo "visible" a propósito: es una decisión de layout de ESE form, no un estilo de componente).

Paso 2 — Page

Ubicación: `\src/pages/\<dominio>\<Entidad>Page.jsx\`. Una Page solo debe:

- 1\ Configurar \useCrudResource\ (endpoint + emptyForm + mapToForm).
- 2\ Renderizar cabecera, buscador, grid y modal con los componentes de \ui/\.
- 3\ Delegar el contenido del formulario al `\\<Entidad>Form\` del paso 1.

```
\\`jsx
```

```
import { Building2, Plus } from 'lucide-react';
import { fieldMapper, useCrudResource } from '../hooks/useCrudResource';
import Modal from '../components/Modal';
import PageHeader from '../components/ui/PageHeader';
import Button from '../components/ui/Button';
import SearchInput from '../components/ui/SearchInput';
```

```

import Alert from '../components/ui/Alert';
import LoadingState from '../components/ui/LoadingState';
import EmptyState from '../components/ui/EmptyState';
import CardGrid from '../components/ui/CardGrid';
import EntityCard from '../components/ui/EntityCard';
import Form from '../components/ui/Form';
import FormActions from '../components/ui/FormActions';
import \<Entidad>Form from './\<Entidad>Form';

const campoVacio = { \<Entidad>Codigo: "", \<Entidad>Nombre: "", \<Entidad>Descripcion: "" };

// Solo hace falta si el form escribe en un nombre de campo distinto al que
// lee la respuesta (ver Paso 0). Si ya coinciden 1:1, se puede omitir
// mapToForm y el hook usa identidad por defecto.

const mapToForm = fieldMapper({
  \<Entidad>Codigo: 'codigo',
  \<Entidad>Nombre: 'nombre',
  \<Entidad>Descripcion: 'descripcion',
});

export default function \<Entidad>Page() {
  const {
    items, loading, error, buscar, setBuscar, cargar,
    modalOpen, editando, form, setForm, erroresForm, guardando,
    abrirCrear, abrirEditar, cerrarModal, guardar, desactivar,
  } = useCrudResource({ endpoint: '\<endpoint>', emptyForm: campoVacio, mapToForm });

  return (
    \<div className="p-6">
      \<PageHeader title="\<Título>" subtitle="\<Dominio> · \<Entidad>">
        \<Button onClick={abrirCrear} icon={Plus}>Nuevo \<entidad>\</Button>
      \</PageHeader>
      \<SearchInput value={buscar} onChange={setBuscar} onSubmit={() => cargar()} />
      \<Alert>{error}\</Alert>
      {loading ? (
        \<LoadingState message="Cargando..." />
      ) : items.length === 0 ? (
        \<EmptyState icon={Building2} message="No hay registros todavía." />
      ) : (
        \<CardGrid>
          {items.map((item) => (
            \<EntityCard

```

```

key={item.id}
initial={item.nombre?.[0]}
active={item.activo}
title={item.nombre}
meta={[item.descripcion]}
footer={item.codigo}
onEdit={() => abrirEditar(item)}
onToggle={item.activo ? () => desactivar(item) : undefined}
/>
)))
</CardGrid>
})
<Modal open={modalOpen} onClose={cerrarModal} title={editando ? 'Editar <entidad>' : 'Nuevo <entidad>'}>
  <Form onSubmit={guardar} generalError={erroresForm.general?.[0]}>
    <<Entidad>Form form={form} setForm={setForm} errors={erroresForm} />
  <FormActions onCancel={cerrarModal} submitting={guardando} />
</Form>
</Modal>
</div>
);
}
\\`

```

Qué config acepta useCrudResource\ (src/hooks/useCrudResource.js\)

| Opción | Obligatoria | Default | Para qué |

|---|---|---|---|

| \endpoint\ | sí | — | Ruta de la API (/microneces\)

| \emptyForm\ | sí | — | Forma inicial del form (crear / limpiar) |

| \mapToForm\ | no | identidad | Item de la lista → forma del form (Paso 0) |

| \idKey\ | no | 'id'\ | Solo si el backend aún no devuelve id\

| \searchParam\ | no | 'buscar'\ | Nombre del query param de búsqueda |

| \buildConfirmMessage\ | no | \¿Desactivar "\${item.nombre}"?\ | Texto del confirm()\ al desactivar |

| \connectionErrorMessage\ / \saveErrorMessage\ / \deactivateErrorMessage\ | no | mensajes genéricos | Solo si este módulo necesita un texto distinto |

El hook nunca tiene un if por entidad: toda diferencia entre módulos entra

por config, no por ramas dentro de useCrudResource.js. Si en algún módulo

sientes que necesitas un if (endpoint === '...') dentro del hook, es señal de

que falta una opción de config, no de que haya que bifurcar el hook.

Paso 3 — Ruta

1\ Verifica que el módulo ya tenga una entrada en `\src/nav/menu.js\` (normalmente sí, porque el menú se generó completo desde el esquema de la base de datos).

2\ En `\src/routes.jsx\`:

- Importa la Page nueva.
- Agrégala al diccionario `\paginasReales\` con el mismo `\path\` que tiene en

`\menu.js\`.

En cuanto el `\path\` está en `\paginasReales\`, `\PlaceholderPage\` deja de mostrarse para esa ruta automáticamente.

Checklist antes de dar el módulo por terminado

`\- []\npx eslint src\` sin errores.

`\- []\npx vite build\` compila sin warnings de imports rotos.

`\- []` Probado en el navegador: crear, editar, buscar, desactivar y cancelar el modal.

`\- []` El mensaje de error 422 se ve bajo el campo correcto (prueba enviando el form vacío si el campo es requerido).

Qué evitar

`\- \*\*No!\*` metas lógica de un módulo específico dentro de `\components/ui/\*` — esos archivos son genéricos a propósito. Lo específico del recurso vive en `\<Entidad>Form.jsx\` y en la config que le pasas a `\useCrudResource\`.

`\- \*\*No!\*` crees un único `\Form.jsx\` gigante con un `\switch\if\` por módulo — la idea es un archivo `\<Entidad>Form.jsx\` pequeño por recurso, no una fábrica universal de formularios.

`\- \*\*No!\*` agregues un `\if\` dentro de `\useCrudResource.js\` para un caso particular — añade una opción de config con un valor por default en su lugar.

`\- \*\*No!\*` repitas clases de Tailwind largas dentro de una Page — si aparece una, probablemente falta un componente en `\components/ui/\`.

usa esta guía para crear un documento pdf para descargar